Cascading Style Sheet & CSS 3

Prepared by Makwana Shailesh

WHAT YOU WILL LEARN

1. What is CSS?
2. Three ways to add CSS
3. Class and ID selectors
4. Pseudo — special states and parts
5. Font properties
6. Text properties
7. Background properties
8. List properties
9. Margin properties
10. CSS comments

1 What is CSS?

CSS stands for **Cascading Style Sheets**. Try to think of it this way: a web page is like a person. The **HTML** is the skeleton — it gives the page its shape and structure (headings, paragraphs, pictures, buttons). The **CSS** is everything else — the clothes, the hairstyle, the colours, the jewellery. It decides how the page actually *looks* when a visitor opens it.

Without CSS, every web page would look exactly the same: plain black text on a white background, with blue underlined links. That was how the early internet looked in the 1990s. CSS is the reason modern websites can be beautiful — with soft shadows, rounded corners, gradients, animated buttons, and layouts that adjust to any screen size.

Simple idea: HTML says *what* is on the page. CSS says *how* it should look.

Why do we use CSS?

- **It keeps things tidy.** The content stays in the HTML file, and all the colours, fonts, and spacing live in the CSS file. Each part does one job.
- **It saves a huge amount of time.** Imagine you have 500 pages on your website, and you want to change every heading from blue to red. Without CSS, you would edit 500 files. With CSS, you change one line in one file, and all 500 pages update instantly.
- **It makes pages load faster.** The browser downloads the CSS file once and remembers it. Every other page on the site loads much faster.
- **It works on every screen.** You can write special CSS rules that only apply on phones, or only on big desktop screens. That's how websites look good everywhere.
- **It makes websites accessible.** With good CSS, you can make text bigger, improve contrast, and help people with poor eyesight read your content more easily.

How a CSS rule is written

A CSS rule always has two main parts: a **selector** (which element to style) and a **block** (a group of instructions inside curly braces). Every instruction inside the block is called a **declaration**.

```
selector {
    property : value;   /* one instruction */
    property : value;   /* another instruction */
}
```

Here is a real example:

```
h1 {
    color: #1e3a8a;
    font-size: 32px;
}
```

Let's break it down line by line:

- `h1` — this is the **selector**. It tells the browser "find every <h1> heading on the page".
- `color` — this is the **property**. It tells the browser what we want to change (in this case, the text colour).
- `#1e3a8a` — this is the **value**. It's a hex code that represents a deep blue.
- `color: #1e3a8a;` — this whole line is one **declaration**. Notice that it ends with a semicolon `;`. Every declaration must end with a semicolon.
- `{ ... }` — the **curly braces** wrap all the declarations that belong to that selector.

The "Cascading" part — which rule wins?

The word *cascading* is important. Sometimes two different rules try to style the same element. For example, a rule might say "make headings blue", and another rule might say "make headings red". The browser has to decide which one wins. It uses a simple priority system called **specificity**.

Priority	Type of rule	Strength
1 (strongest)	`!important`	Beats everything
2	Inline style `style="..."`	1000
3	ID selector `#id`	100
4	Class or pseudo-class `.class`	10
5 (weakest)	Element `p`	1

Easy tip: if two rules have the exact same strength, the one written **later** in the stylesheet wins. So the order you write things in really does matter.

2 Three ways to add CSS

You can attach CSS to an HTML page in three different ways. Each one has its own strengths and weaknesses. Let's look at all three so you know when to use which.

A) Inline CSS — inside the tag itself

Inline CSS is written directly *inside the HTML tag* using a special attribute called `style`. It only affects that one single element and nothing else.

```
<p style="color: red; font-size: 20px;">
    This paragraph is styled inline.
</p>
```

LIVE DEMO

This paragraph is styled inline.

Good side: it's quick. If you just want to test something or override one specific element, this works instantly. It also has very high priority, so it usually wins over other rules.

Bad side: it's messy. Your HTML file gets filled with style information. It's also impossible to reuse — if you want ten paragraphs to look the same, you have to copy the style ten times. This makes the code hard to maintain. Most professional developers avoid inline CSS except for very small one-off tweaks.

B) Internal CSS — inside the <style> tag

Internal CSS is written inside a special `<style>` tag, which itself sits inside the `<head>` section of the HTML file. It affects the whole page, but only that one page.

```
<head>
    <style>
        p {
            color: green;
            font-size: 18px;
        }
    </style>
</head>
```

This entire page you are reading right now uses **internal CSS**! Scroll to the top of the source code and you will see a big `<style>` block with all the styling in it.

Good side: everything is in one place. You don't need extra files. It's a good choice for small projects, single-page sites, or when you want to test some styles quickly.

Bad side: it only applies to one HTML file. If your website has many pages, you'd have to copy this block into every single one. That's a maintenance nightmare.

C) External CSS — a separate .css file

External CSS is written in a completely separate file that ends with `.css`. You then link that file to your HTML using a `<link>` tag. This is the **best and most professional** way to style a website.

```
<!-- In your HTML file -->
<head>
    <link rel="stylesheet" href="style.css">
</head>

/* In your style.css file */
p {
    color: blue;
    font-size: 18px;
}
```

You can even import one CSS file from inside another CSS file. This helps you organise big projects into small, manageable pieces:

```
@import url("colors.css");
@import url("layout.css");
@import url("buttons.css");
```

Good side: this is reusable across hundreds of pages. You write it once, link it everywhere, and every page instantly shares the same look. If you want to change the colour of all buttons across the entire site, you change one line in one file.

Bad side: it requires an extra network request the first time a page loads (though after that, the browser caches it). For very small single-page projects, it can feel like overkill.

Which one should you choose?

Type	Where it lives	Affects	Reusable?
Inline	`style=""` attribute	One element	No
Internal	`<style>` in head	One page	No
External	A `.css` file	Whole website	Yes ✅

Rule of thumb: Use **external** CSS for real projects, **internal** for quick tests on a single page, and **inline** only when you really have no other choice.

3 Class and ID selectors

Now that we know how to add CSS, we need to know how to *target* specific elements. That's where selectors come in. Two of the most important selectors are **class** and **ID**. They look similar, but they behave very differently.

A) The class selector `.classname`

A class is like a label you can stick onto any element. You give it a name (starting with a dot in the CSS) and then use the same label on as many elements as you like. This makes classes perfect for styling groups of elements that should all look the same.

- In CSS, a class selector starts with a **dot** `.`
- In HTML, you attach it using the `class` attribute.
- A single element can have **several classes** at once: `class="box red big"`. The browser applies all of them.

```
<p class="highlight">Highlighted text</p>

/* CSS */
.highlight {
    background-color: #fde68a;
    padding: 2px 6px;
}
```

LIVE DEMO — CLASS

Normal text. This text has class="highlight".

This paragraph has class="box-style".

This one has BOTH classes: "highlight box-style".

B) The ID selector `#idname`

An ID is like a unique name for one specific element. Just like in real life, a person has an ID number that belongs only to them, an ID in CSS should only be used for one element on the page.

- In CSS, an ID selector starts with a **hash** `#`
- In HTML, you attach it using the `id` attribute.
- Each ID must be **unique** — never use the same ID twice on a page.
- IDs are also used by JavaScript and for jump links (like `#section1`).

```
<h2 id="main-title">Welcome</h2>

/* CSS */
#main-title {
    color: #be123c;
    font-size: 1.8rem;
    font-weight: bold;
}
```

LIVE DEMO — ID

Welcome (styled with #main-title)

C) Combining selectors

You can combine class and ID selectors to be more specific about what you want to target:

```
p.highlight      /* only <p> elements that have class "highlight" */
div#main-title   /* a <div> whose id is "main-title" */
.highlight.box   /* an element having BOTH classes at the same time */
```

D) Class vs ID — the simple difference

Feature	Class `.`	ID `#`
Symbol used	A dot `.`	A hash `#`
Can be reused	Yes — on many elements	No — only one element per page
Multiple per element	Yes (space-separated)	No
CSS strength	10	100
Best used for	Styling groups of things	Unique blocks or JS hooks

Simple memory: a **class** is like a T-shirt that many people can wear. An **ID** is like a passport number — it belongs to exactly one person.

4 Pseudo — special states and parts

Imagine you want a button to change colour when someone hovers over it. The colour change is not part of the button's normal appearance — it only happens in a specific *state*. In CSS, we describe these states using **pseudo-classes**.

Similarly, sometimes you want to style just the first letter of a paragraph, or add something before or after an element. These are handled by **pseudo-elements**.

- **Pseudo-class** — written with **one colon** `:` — describes a *state* like hover, focus, or "is the first child".
- **Pseudo-element** — written with **two colons** `::` — describes a *part* of an element like the first letter, the first line, or content added before/after.

A) Common pseudo-classes

Pseudo-class	What it means
`:hover`	Mouse is currently over the element
`:active`	Element is being clicked right now
`:focus`	Element has keyboard or input focus
`:link`	An unvisited link
`:visited`	A link that has already been visited
`:first-child`	The first child inside its parent
`:last-child`	The last child inside its parent
`:nth-child(n)`	The n-th child (can use `odd`, `even`)
`:not(selector)`	Elements that do NOT match the selector
`:checked`	A checkbox or radio button that is checked

LIVE DEMO — HOVER AND ACTIVE

Hover over me Click me

The buttons lift up and change colour when you hover, and turn red when you click.

```
.btn {
    background: #3b82f6;
    color: white;
    padding: 8px 16px;
    border: none;
    border-radius: 6px;
    cursor: pointer;
}
.btn:hover  { background: #8b5cf6; }   /* mouse over */
.btn:active { background: #ef4444; }   /* being clicked */
```

LIVE DEMO — LINK STATES

Hover over this link — it is blue when unvisited, purple once visited, and red when you hover it.

```
a:link    { color: #2563eb; text-decoration: none; }
a:visited { color: #7c3aed; }
a:hover   { color: #dc2626; text-decoration: underline; }
```

LIVE DEMO — FIRST-CHILD, LAST-CHILD, NTH-CHILD

- Apple — first child (green & bold)
- Banana — 2nd child (blue background)
- Cherry — middle child
- Mango — last child (red & bold)

```
.fruit-list li:first-child { color: #16a34a; font-weight: bold; }
.fruit-list li:last-child  { color: #dc2626; font-weight: bold; }
.fruit-list li:nth-child(2) { background: #dbeafe; }
```

LIVE DEMO — NOT() AND CHECKED

- Task item (styled)
- Skipped item (not styled, because of :not(.skip))
- Task item (styled)

☑ I am checked — strikethrough & green

☐ I am unchecked — normal

```
.not-demo li:not(.skip) { color: #0f766e; }
input[type="checkbox"]:checked + label {
    color: #16a34a;
    text-decoration: line-through;
}
```

B) Common pseudo-elements

Pseudo-element	What it means
`::before`	Adds content before the element's content
`::after`	Adds content after the element's content
`::first-letter`	The very first letter of a block
`::first-line`	The first line of a block
`::selection`	Text the user has highlighted
`::marker`	The bullet or number of a list item

LIVE DEMO — ::BEFORE AND ::AFTER

✨ CSS makes the web beautiful ✨

```
.quote::before { content: "✨ "; font-size: 1rem; color: #f59e0b; }
.quote::after  { content: " ✨"; font-size: 1rem; color: #f59e0b; }
```

LIVE DEMO — ::FIRST-LETTER, ::FIRST-LINE, ::SELECTION

This paragraph uses ::first-letter to create a beautiful drop cap, exactly like in a printed magazine. THIS PARAGRAPH USES ::FIRST-LINE. ONLY THE FIRST LINE OF THIS BLOCK WILL BE BOLD, and all in uppercase — no matter how the text wraps inside the box.

Try selecting this sentence with your mouse — the ::selection pseudo-element gives it a soft yellow highlight.

```
.dropcap::first-letter {
    font-size: 2.8rem;
    color: #be123c;
    font-weight: bold;
    float: left;
}
.firstline::first-line {
    font-weight: bold;
    color: #1e40af;
    text-transform: uppercase;
}
::selection {
    background: #fde68a;
    color: #78350f;
}
```

Easy memory: a pseudo-class describes a **state** (What is happening to the element right now?). A pseudo-element describes a **part** (Which specific piece of the element?).

5 Font properties

Font properties control the **typeface and appearance** of text. This is where you decide whether your headings look formal and old-fashioned (like a book), or modern and clean (like a tech company website).

Property	What it does	Example values
`font-family`	Sets the typeface	`Georgia, serif`
`font-size`	Sets the size	`16px`, `1.2rem`
`font-weight`	Thickness of the letters	`normal`, `bold`, `100–900`
`font-style`	Italic slant	`normal`, `italic`, `oblique`
`font-variant`	Small caps style	`normal`, `small-caps`
`line-height`	Space between lines	`1.6`, `24px`
`font`	A short way to write all of them	`italic bold 18px/1.5 Georgia`

font-family — choosing the typeface

The `font-family` property decides which "shape of letters" the text uses. Because not every computer has every font installed, you should always give a **list of fonts**. The browser tries the first one; if it's not available, it tries the next, and so on. Always end the list with a generic family as a safety net.

- `serif` — fonts with little "feet" at the ends of letters (like Georgia).
- `sans-serif` — clean fonts without feet (like Arial or Verdana).
- `monospace` — fonts where every letter is the same width (like Courier).

```
p {
    font-family: Georgia, "Times New Roman", serif;
}
```

LIVE DEMO — FONT-FAMILY

Georgia (serif) — classic and readable.

Courier New (monospace) — code-like.

Verdana (sans-serif) — clean and modern.

font-size — how big the text is

You can set the font size using several different units. The three most common are:

- `px` (pixels) — an exact size, like `16px`. Good for precision, but doesn't respect the user's browser settings.
- `rem` — relative to the root font size (usually 16px). `1.2rem` means "1.2 times the base size". This is the friendliest unit for accessibility.
- `%` — a percentage of the parent element's font size.

```
.small  { font-size: 12px; }
.normal { font-size: 1rem; }     /* 16px by default */
.big    { font-size: 150%; }
```

LIVE DEMO — FONT-SIZE

12px text — small note

1rem text (16px by default) — normal reading size

150% text — noticeably bigger

font-weight — how thick the letters are

This controls how **bold** the text is. You can use keywords like `normal` and `bold`, or numbers from 100 (very thin) to 900 (very thick). Not every font supports every weight, but most support at least 300, 400, 700 and 900.

```
.light { font-weight: 300; }
.bold  { font-weight: 700; }
.heavy { font-weight: 900; }
```

LIVE DEMO — FONT-WEIGHT

font-weight: 300 — light and thin

font-weight: 700 — a nice strong bold

font-weight: bold — the classic keyword

font-style — italic slanted text

This property makes text slanted. `italic` uses the font's true italic design, while `oblique` simply tilts the normal letters.

```
.italic  { font-style: italic; }
.oblique { font-style: oblique; }
```

font-variant — small caps

`small-caps` shows lowercase letters as small capitals. It is a subtle, elegant effect often used in book titles and business cards.

LIVE DEMO — FONT-STYLE & FONT-VARIANT

This text is italic.

This text is oblique.

THIS TEXT USES SMALL CAPS VARIANT.

The font shorthand

The `font` property lets you set many font properties in one line. This is a big time-saver, but there is a rule: you **must** include `font-size` and `font-family`, and they must be in that order. The line-height comes right after the size, separated by a slash.

```
/* font: style weight size/line-height family; */
font-short {
    font: italic 700 18px/1.5 Georgia, serif;
}
```

LIVE DEMO — FONT SHORTHAND

This paragraph uses one short line: italic 700 18px/1.5 Georgia, serif

Important: if you write the shorthand wrong — for example, missing the size or family — the entire line is ignored by the browser. So be careful with the order.

6 Text properties

Font properties decide how letters *look*. Text properties decide how those words are **arranged on the page** — whether they are centred, spaced out, underlined, or transformed into uppercase.

Property	What it does
`color`	The colour of the text
`text-align`	left / right / center / justify
`text-decoration`	underline / overline / line-through / none
`text-transform`	uppercase / lowercase / capitalize
`text-indent`	Indents the first line of a paragraph
`letter-spacing`	Space between individual letters
`word-spacing`	Space between words
`line-height`	Vertical space between lines
`text-shadow`	Adds a shadow behind the text

color — setting the text colour

You can write colours in many different ways. All of the following examples produce a deep pink:

```
p { color: red; }                      /* a keyword name */
p { color: #be123c; }                  /* a hex code */
p { color: rgb(190, 18, 60); }         /* red, green, blue */
p { color: rgba(190, 18, 60, 0.8); }   /* with transparency */
p { color: hsl(346, 83%, 41%); }       /* hue, saturation, lightness */
```

text-align — where the text sits

LIVE DEMO — TEXT-ALIGN

Centered text (text-align: center)

Right-aligned text (text-align: right)

Justified text stretches so that both the left and right edges are perfectly straight, just like in a newspaper or a book. This is great for long paragraphs of reading material.

text-transform — changing letter case

This property lets you change the case of the text without editing the HTML. Very useful for headings and menus.

LIVE DEMO — TEXT-TRANSFORM

THIS WAS TYPED IN LOWERCASE

this was typed in uppercase

Capitalize Each Word Of This Sentence

text-decoration — lines under, over, or through text

LIVE DEMO — TEXT-DECORATION

Underlined text

Overlined text

Struck-through text

A link with text-decoration: none

text-indent, letter-spacing, word-spacing

LIVE DEMO — INDENT & SPACING

This paragraph has a first line indent of 40px, just like a printed book. Only the first line is pushed in — the rest of the lines start at the normal left edge.

LETTER SPACING

These words are far apart

line-height and text-shadow

`line-height` controls the vertical space between lines of text. A value around `1.5` or `1.6` is very comfortable for reading. Anything under `1.2` starts to feel cramped.

LIVE DEMO — LINE-HEIGHT & TEXT-SHADOW

This paragraph has a line-height of 2.2, which makes the lines sit far apart and gives a very relaxed reading experience. Line height is one of the most important properties for readability on the web.

Text with a soft shadow

```
.ti-indent  { text-indent: 40px; }
.ls-spacing { letter-spacing: 6px; }
.ws-wordsp  { word-spacing: 18px; }
.lh-lh      { line-height: 2.2; }
.ts-shadow  { text-shadow: 2px 2px 4px rgba(0,0,0,0.3); }
```

text-shadow format: `text-shadow: horizontal-offset vertical-offset blur-radius color;` For example, `2px 2px 4px black` pushes the shadow 2 pixels right, 2 pixels down, with a 4-pixel blur.

7 Background properties

Background properties control what appears *behind* an element's content. You can use a solid colour, a smooth gradient, or even a full picture. Backgrounds are one of the easiest ways to make a page feel rich and designed.

Property	What it does
`background-color`	Sets a solid colour behind the element
`background-image`	Puts an image or gradient behind the content
`background-repeat`	repeat / repeat-x / repeat-y / no-repeat
`background-position`	Where to place the image (e.g. center)
`background-size`	auto / cover / contain / exact size
`background-attachment`	scroll / fixed
`background`	A short way to write all of the above

background-color

LIVE DEMO — BACKGROUND-COLOR

This box has background-color: #fde68a

```
.bg-color { background-color: #fde68a; }
```

background-image (with gradients)

A **gradient** is a smooth blend between two or more colours. In CSS, gradients are actually created with `background-image`, not `background-color`, because they count as an image.

LIVE DEMO — LINEAR GRADIENT

background-image: linear-gradient(135deg, #1e3a8a, #22d3ee)

```
.bg-gradient {
    background-image: linear-gradient(135deg, #1e3a8a, #6d28d9, #22d3ee);
}
```

background-repeat

By default, a background image repeats in both directions to fill the entire area. You can control this with `background-repeat`.

LIVE DEMO — REPEAT VS NO-REPEAT

```
.bg-repeat {
    background-image: url("pattern.png");
    background-repeat: repeat;          /* tiles in both directions */
    background-size: 24px 24px;
}
.bg-no-repeat {
    background-image: url("dot.png");
    background-repeat: no-repeat;
    background-position: center;        /* centers the image */
    background-size: 40px 40px;
}
```

background-size and background-position

`background-size: cover` stretches the image so it completely covers the box — even if some parts get cropped. `contain` does the opposite: it fits the whole image inside the box, which may leave empty space.

LIVE DEMO — BACKGROUND-SIZE: COVER

background-size: cover;

The gradient fills the whole box beautifully. Try resizing the browser — it always stays centred.

```
.bg-cover {
    background-image: url("photo.jpg");
    background-size: cover;          /* fills the box */
    background-position: center;     /* keeps the middle visible */
    background-repeat: no-repeat;
}
```

The background shorthand

The `background` property lets you write many things in one line. The trick is to put all the values in one line, separated by a space:

```
/* background: color image repeat position / size; */
.box {
    background: #1e293b url("bg.png") no-repeat center / cover;
}
```

LIVE DEMO — BACKGROUND SHORTHAND

This box uses the shorthand background: #1e293b url("none") no-repeat center / cover

Easy memory: background properties are like painting a wall. `background-color` is the base paint, `background-image` is the wallpaper, and `background-size` is how big you make the wallpaper.

8 List properties

Lists are everywhere on the web — navigation menus, feature lists, step-by-step guides. CSS gives us fine control over how list items look: the bullet style, the position, and even custom images.

Property	What it does	Values
`list-style-type`	The bullet or number type	`disc`, `circle`, `square`, `decimal`, `upper-roman`, `lower-alpha`, `none`
`list-style-position`	Whether the marker sits inside or outside the text block	`inside`, `outside`
`list-style-image`	Use a picture as the bullet	`url("star.png")`, `none`
`list-style`	A short way to write all three	`square inside`

list-style-type — the shape of the bullet

LIVE DEMO — LIST-STYLE-TYPE

- disc (default for <ul>)
- Second item
- circle
- Second item
- square
- Second item
- none — no bullets at all
- Second item
- I. upper-roman
- II. Second item
- a. lower-alpha
- b. Second item

```
.list-disc   { list-style-type: disc; }
.list-circle { list-style-type: circle; }
.list-square { list-style-type: square; }
.list-none   { list-style-type: none; }
.list-roman  { list-style-type: upper-roman; }
.list-alpha  { list-style-type: lower-alpha; }
```

list-style-position — inside or outside

With **outside** (the default), the bullet sits in the margin, and the text wraps beside it. With **inside**, the bullet becomes part of the text, so wrapped lines start underneath the bullet.

LIVE DEMO — INSIDE VS OUTSIDE

inside — the bullet is part of the text flow

- First item with a longer line of text so you can clearly see how the second line wraps under the bullet when using list-style-position: inside.
- Second item

outside (default) — the text wraps beside the bullet

- First item with a longer line of text so you can clearly see how the second line aligns with the first line of text, not with the bullet.
- Second item

```
.list-inside  { list-style-position: inside; }
.list-outside { list-style-position: outside; }
```

Styling the marker with ::marker

LIVE DEMO — ::MARKER

- The marker is red, bold and slightly larger
- Second item
- Third item

```
.list-custom li::marker {
    color: #dc2626;
    font-weight: bold;
    font-size: 1.3em;
}
```

list-style-image

```
ul {
    list-style-image: url("star.png");
    /* if the image fails to load, list-style-type is used as fallback */
}
```

The list-style shorthand

```
ul {
    list-style: square inside;
}
/* Order does not matter, but all three may be given:
   list-style: square inside url("star.png"); */
```

Real-world example: navigation menus almost always start with `list-style: none;` to remove the bullets, then style the `<li>` elements as horizontal buttons. So this knowledge here is used in virtually every website you visit.

9 Margin properties

Margin is the transparent space *outside* an element's border. It pushes other elements away. It is one of the most important tools for creating a clean, uncluttered layout.

People often confuse **margin** with **padding**. Here is the simplest way to remember the difference:

- **Padding** is the space *inside* the border, between the border and the content. The background colour shows through padding.
- **Margin** is the space *outside* the border. It is always transparent and just pushes other things away.

Property	What it controls
`margin-top`	Space above the element
`margin-right`	Space to the right
`margin-bottom`	Space below the element
`margin-left`	Space to the left
`margin`	A short way to write all four

The margin shorthand — 4 ways

Code	Meaning
`margin: 20px;`	All four sides = 20px
`margin: 10px 30px;`	top/bottom = 10px, left/right = 30px
`margin: 10px 20px 30px;`	top = 10px, sides = 20px, bottom = 30px
`margin: 10px 20px 30px 40px;`	top, right, bottom, left (clockwise)

Memory trick: the four-value shorthand goes **clockwise starting from the top**: Top → Right → Bottom → Left. Think of it like a clock face.

LIVE DEMO — 1 VALUE

margin: 20px (all sides)

LIVE DEMO — 2 VALUES

margin: 10px 30px (top/bottom 10, left/right 30)

LIVE DEMO — 3 VALUES

margin: 10px 20px 30px (top 10, sides 20, bottom 30)

LIVE DEMO — 4 VALUES

margin: 10px 20px 30px 40px (TRBL)

Auto margins — the classic "center a box" trick

If you set the left and right margins to `auto` on a block element that has a fixed width, the browser will automatically split the remaining space equally and center the element. This is one of the oldest tricks in CSS.

LIVE DEMO — MARGIN: 0 AUTO

I am centered with margin: 0 auto

```
.m-auto {
    width: 300px;
    margin: 0 auto;   /* top/bottom = 0, left/right = auto */
}
```

Margins can overlap (margin collapse)

When the bottom margin of one block meets the top margin of the next block, they **do not add up**. The larger one wins. This is called **margin collapsing**, and it surprises many beginners.

```
/* Element A: margin-bottom: 30px; */
/* Element B: margin-top: 20px; */
/* The gap between them is 30px, NOT 50px. */
```

This only happens with top and bottom margins in normal block flow. It does not happen with flexbox, CSS grid, floated elements, or absolutely positioned elements.

Margin vs Padding — quick comparison

Margin	Padding
Space **outside** the border	Space **inside** the border
Always transparent	Shows the background colour
Can be negative	Cannot be negative
Can collapse vertically	Never collapses

10 CSS comments

A comment is a note for humans. The browser completely ignores it. Comments are incredibly useful — they help explain your code to your teammates, to your future self, and to anyone who reads your work later.

In CSS, a comment starts with `/*` and ends with `*/`. Anything between those two markers is completely ignored by the browser.

Single-line comment

```
/* This styles the main heading */
h1 {
    color: #1e3a8a;   /* dark blue */
}
```

Multi-line comment

```
/*
   ================================
   HEADER SECTION STYLES
   Created : 2024
   Author  : Your Name
   ================================
*/
header {
    background: #1e293b;
    color: white;
}
```

Commenting out code temporarily

One of the most useful uses of comments is to temporarily disable a line of CSS while debugging, without actually deleting it.

```
h1 {
    color: blue;
    /* color: red; */    ← disabled for now
    font-size: 32px;
}
```

Rules to remember:

- CSS comments use `/* ... */` only.
- Comments **cannot be nested**. The first `*/` closes the comment.
- Comments work anywhere in a stylesheet — between rules, inside a block, even between values.
- Comments cannot be used inside a `style=""` attribute.
- HTML uses `<!-- ... -->`, CSS uses `/* ... */`.

Bad vs Good example

```
/* ❌ BAD — nested comments break */
/* outer comment
   /* inner comment */
   still inside? NO — already closed
*/

/* ✅ GOOD — use a different marker inside */
/* outer comment
   -- inner note here --
   the comment is still open
*/
```

Why comment your CSS?

- Explains *why* a rule exists — the "why" is often not obvious from the code.
- Separates large stylesheets into clear, labelled sections.
- Helps teammates (and future you) understand the code faster.
- Useful for temporarily disabling a rule while debugging.

Best practice: Don't over-comment. Explaining *why* `margin-top: -8px;` is used is valuable. But writing `/* sets the colour */` above `color: red;` is just noise.

WHAT YOU WILL LEARN

1. Border Property
2. Background & Gradient
3. Drop Shadow
4. 2D & 3D Transform
5. Transition
6. Box Sizing
7. Position

1 Border Property

A **border** is the line that goes around an element. Think of it like a picture frame around a photo. Borders help separate things visually and make boxes feel neat and touchable.

A border has three parts: a **width** (how thick it is), a **style** (what kind of line it is), and a **colour**. You can write all three in one line:

```
/* border: width style color; */
.box {
    border: 3px solid #1e3a8a;
}
```

Simple memory: a border always needs a style. If you write a width and a colour but forget the style, the border will not appear at all.

All the border styles

CSS gives you many border styles. Here are the most common ones, all shown live below so you can compare them side by side.

LIVE DEMO — BORDER STYLES

solid dashed dotted double

groove ridge inset outset

```
.solid  { border: 4px solid #3b82f6; }
.dashed { border: 4px dashed #8b5cf6; }
.dotted { border: 4px dotted #ec4899; }
.double { border: 6px double #10b981; }
.groove { border: 6px groove #f59e0b; }
.ridge  { border: 6px ridge #ef4444; }
.inset  { border: 6px inset #14b8a6; }
.outset { border: 6px outset #6366f1; }
```

Individual side borders

You don't have to put a border on all four sides. You can target each side separately:

```
.box {
    border-top:    3px solid #3b82f6;
    border-right:  3px dashed #8b5cf6;
    border-bottom: 3px dotted #ec4899;
    border-left:   3px double #10b981;
}
```

border-radius — rounded corners

Before CSS3, every box on the web had sharp 90-degree corners. The `border-radius` property changed all of that. It lets you round the corners of any element — even making perfect circles.

LIVE DEMO — BORDER-RADIUS

No radius

12px

30px

50% (circle)

50% 0 50% 0

20px 0px 20px 0px

```
.round-0  { border-radius: 0; }
.round-12 { border-radius: 12px; }   /* soft corners */
.round-30 { border-radius: 30px; }   /* very round */
.round-50 { border-radius: 50%; }    /* perfect circle (if width = height) */
.round-leaf { border-radius: 50% 0 50% 0; }  /* leaf shape */
.round-mix  { border-radius: 20px 0px 20px 0px; }  /* any shape */
```

Just like margins, `border-radius` can take 1, 2, 3, or 4 values. The order is **top-left → top-right → bottom-right → bottom-left** (clockwise, starting from the top-left corner).

border-image — a picture as a border

CSS3 also introduced `border-image`, which lets you use a gradient or an image as your border. The `1` at the end tells the browser how to slice the image.

LIVE DEMO — BORDER-IMAGE WITH GRADIENT

This box uses a gradient border-image

```
.bd-image {
    border: 6px solid transparent;
    border-image: linear-gradient(45deg, #f43f5e, #8b5cf6, #06b6d4) 1;
}
```

Important: the border must first be set with a width and a transparent style, otherwise the border-image has no space to appear.

2 Background & Gradient Property

A **background** is what sits behind the content of an element. You already know you can give a simple colour. But CSS3 added something much more exciting: **gradients** — smooth blends between colours, created by the browser itself.

Why gradients matter: before CSS3, if you wanted a gradient you had to make it in Photoshop and save it as an image file. Now you can write one line of CSS and get a perfect, sharp gradient that scales to any size.

Linear gradient — a straight blend

A **linear gradient** blends colours in a straight line. You choose a direction and then a list of colours.

LIVE DEMO — LINEAR GRADIENTS

linear-gradient(to right, #7c3aed, #ec4899)

linear-gradient(to bottom, #0ea5e9, #22d3ee)

linear-gradient(135deg, #f59e0b, #ef4444, #8b5cf6)

```
.lg-right  { background: linear-gradient(to right, #7c3aed, #ec4899); }
.lg-bottom { background: linear-gradient(to bottom, #0ea5e9, #22d3ee); }
.diagonal  { background: linear-gradient(135deg, #f59e0b, #ef4444, #8b5cf6); }
```

Directions can be written in words (`to right`, `to bottom`) or as an angle (`135deg`). You can also use more than two colours — the browser blends them evenly.

Radial gradient — a circular blend

A **radial gradient** starts from a point and spreads outward in a circle. This creates a lovely spotlight effect.

LIVE DEMO — RADIAL GRADIENT

radial-gradient(circle, #fde68a, #f59e0b, #b45309)

```
.radial {
    background: radial-gradient(circle, #fde68a, #f59e0b, #b45309);
}
```

Conic gradient — a colour wheel

A **conic gradient** sweeps colours around a circle, like the hands of a clock. It is perfect for pie charts and colourful badges.

LIVE DEMO — CONIC GRADIENT

conic-gradient(from 45deg, #7c3aed, #ec4899, #f59e0b, #22d3ee, #7c3aed)

WHAT YOU WILL LEARN

1. Media Query — making pages responsive
2. Flexbox — the complete layout system
3. Advanced CSS properties

1 Media Query

A **media query** is a small rule you write in your CSS that says: "only apply these styles *when* the screen is a certain size (or type)." This is exactly how one single HTML file can look great on a phone, a tablet, and a big desktop monitor all at the same time.

Think of it like a photo frame on your wall. On a small wall, you use a small frame. On a big wall, you hang a big frame. The picture is the same — only the frame changes to fit. Media queries do the same thing: change the layout to fit the "wall" it happens to be on.

The big idea: you write your normal CSS for the biggest screen first, then use media queries to adjust things for smaller screens. This is called the **desktop-first** approach. You can also do the opposite — **mobile-first** — starting with the smallest screen and using `min-width` to add features as the screen grows.

The basic syntax

```
/* Only apply these styles when the screen is 600px wide or less */
@media (max-width: 600px) {
    body {
        font-size: 14px;
    }
    .menu {
        display: none;
    }
}
```

Every media query has three parts:

- **@media** — the keyword that tells the browser "this is a media query".
- **The condition** — written inside round brackets. For example `(max-width: 600px)` means "apply this when the screen is 600 pixels wide or smaller".
- **The rule block** — the styles inside the curly braces. These only take effect when the condition is true.

Live Demo — try resizing your browser window

The box below has three different rules written for it: one for desktop, one for tablet, one for phone. Watch it change as you resize the browser window or rotate your phone.

LIVE DEMO — RESPONSIVE BOX

I am the same HTML element, but I change my look at different widths!

Drag the edge of your browser window to see me change colour and text.

```
/* Default (desktop) */
.mq-responsive {
    background: #f0f9ff;
    border: 3px dashed #0284c7;
    color: #0c4a6e;
}
.mq-responsive::after { content: " - Desktop view"; }

/* Tablet */
@media (max-width: 900px) {
    .mq-responsive {
        background: #fef3c7;
        border-color: #f59e0b;
        color: #92400e;
    }
    .mq-responsive::after { content: " - Tablet view"; }
}

/* Phone */
@media (max-width: 600px) {
    .mq-responsive {
        background: #fce7f3;
        border-color: #ec4899;
        color: #9d174d;
    }
    .mq-responsive::after { content: " - Phone view"; }
}
```

Live Demo — a responsive card grid

This is the most common real-world use of media queries. The cards start as 4 columns on a big screen, become 2 columns on a tablet, and stack into 1 column on a phone.

LIVE DEMO — 4 / 2 / 1 COLUMNS

Card 1 Card 2 Card 3 Card 4 Card 5 Card 6 Card 7 Card 8

```
.mq-grid {
    display: grid;
    grid-template-columns: repeat(4, 1fr);   /* 4 columns by default */
    gap: 10px;
}

@media (max-width: 900px) {
    .mq-grid { grid-template-columns: repeat(2, 1fr); }   /* 2 columns */
}

@media (max-width: 600px) {
    .mq-grid { grid-template-columns: 1fr; }              /* 1 column */
}
```

Common media features you can check

Feature	What it checks	Example
`max-width`	Screen is at most this wide	`(max-width: 600px)`
`min-width`	Screen is at least this wide	`(min-width: 768px)`
`orientation`	Portrait or landscape	`(orientation: landscape)`
`max-height`	Screen is at most this tall	`(max-height: 500px)`
`hover`	Device supports hovering	`(hover: hover)`
`prefers-color-scheme`	User prefers dark or light mode	`(prefers-color-scheme: dark)`
`prefers-reduced-motion`	User asked for less animation	`(prefers-reduced-motion: reduce)`

Live Demo — orientation

LIVE DEMO — ORIENTATION CHECK

The colour of this box depends on your screen orientation.

On a phone, rotate your device to see it change.

```
@media (orientation: landscape) {
    .mq-orient { background: linear-gradient(135deg, #10b981, #06b6d4); }
}
@media (orientation: portrait) {
    .mq-orient { background: linear-gradient(135deg, #f43f5e, #a855f7); }
}
```

Combining two conditions with and

```
/* Apply only when the screen is between 600px and 900px wide */
@media (min-width: 600px) and (max-width: 900px) {
    .sidebar { display: none; }
}
```

Common breakpoints used in real projects

Device	Typical screen width	Media query to write
Small phone	up to 480px	`(max-width: 480px)`
Large phone	481px – 600px	`(max-width: 600px)`
Tablet portrait	601px – 768px	`(max-width: 768px)`
Tablet landscape	769px – 1024px	`(max-width: 1024px)`
Laptop / Desktop	1025px and up	no query needed (default styles)

Best practice: don't blindly copy breakpoints from other sites. Add a breakpoint wherever *your* design starts to look broken. Shrink your browser slowly and watch for the moment it breaks.

Dark mode in a single query

```
@media (prefers-color-scheme: dark) {
    body {
        background: #0f172a;
        color: #e2e8f0;
    }
}
```

This one query automatically gives your website a dark theme for any user whose device is set to dark mode. It is polite, modern, and takes just seconds to add.

2 Flexbox — the complete layout system

Flexbox is the modern way to arrange items in a row or a column. Before flexbox, centering a box or lining up columns took all kinds of tricks — floats, tables, and negative margins. Flexbox made layouts simple, predictable, and finally fun.

The idea is simple: you make a **flex container**, and everything inside it becomes a **flex item**. Then you use a handful of properties on the container to control how those items are arranged.

Two axes to remember: flexbox works on two axes. The **main axis** is the direction the items flow (left-to-right by default). The **cross axis** is the perpendicular direction (top-to-bottom by default). Almost every flex property controls one axis or the other.

display: flex — turning on the container

The very first thing you do is set `display: flex` on the parent. The instant you do, its children line up in a row instead of stacking on top of each other.

LIVE DEMO — DISPLAY: FLEX

1
2
3

```
.container {
    display: flex;    /* turns on flexbox for all direct children */
}
```

flex-direction — which way the items flow

This property decides the direction of the **main axis**. There are four possible values:

- `row` — left to right (the default)
- `row-reverse` — right to left
- `column` — top to bottom
- `column-reverse` — bottom to top

LIVE DEMO — ROW (DEFAULT)

1 2 3

LIVE DEMO — ROW-REVERSE

1 2 3

LIVE DEMO — COLUMN

1 2 3

LIVE DEMO — COLUMN-REVERSE

1 2 3

```
.row            { flex-direction: row; }
.row-reverse    { flex-direction: row-reverse; }
.column         { flex-direction: column; }
.column-reverse { flex-direction: column-reverse; }
```

Important: when you switch to `column`, the main axis becomes vertical. That means `justify-content` now controls vertical spacing, and `align-items` controls horizontal alignment. The two properties effectively swap jobs.

flex-wrap — what happens when items run out of room

By default, flexbox tries to squeeze everything onto one single line — even if the items become very narrow and ugly. You can change this with `flex-wrap`, which lets items move onto a new line when they run out of space.

- `nowrap` — everything stays on one line (the default)
- `wrap` — items move to a new line when needed
- `wrap-reverse` — same as wrap, but new lines go above

LIVE DEMO — FLEX-WRAP: NOWRAP (DEFAULT)

One Two Three Four Five Six

LIVE DEMO — FLEX-WRAP: WRAP (MOVES TO A NEW LINE)

One Two Three Four Five Six

LIVE DEMO — FLEX-WRAP: WRAP-REVERSE

One Two Three Four Five Six

```
.nowrap       { flex-wrap: nowrap; }
.wrap         { flex-wrap: wrap; }
.wrap-reverse { flex-wrap: wrap-reverse; }
```

flex-flow — a shortcut for both

`flex-flow` combines `flex-direction` and `flex-wrap` into one short line. The first value is the direction, the second is the wrap.

```
/* flex-flow: direction wrap; */
.container-1 {
    flex-flow: row wrap;          /* row direction, wrapping allowed */
}
.container-2 {
    flex-flow: column nowrap;     /* column direction, no wrapping */
}
.container-3 {
    flex-flow: row-reverse wrap;  /* reversed row, wrapping allowed */
}
```

Handy memory: `flex-flow` is just a shortcut that saves you writing two properties separately — the result is identical.

justify-content — spacing along the main axis

This is one of the most useful flexbox properties. It controls how the items are spaced out along the **main axis** (left to right in a row by default).

Value	What it does
`flex-start`	Pushes items to the start (default)
`flex-end`	Pushes items to the end
`center`	Centers all the items
`space-between`	First and last touch the edges; equal space between
`space-around`	Equal space around each item (half-spaces at edges)
`space-evenly`	Perfectly equal gaps everywhere, including the ends

LIVE DEMO — FLEX-START

1 2 3

LIVE DEMO — FLEX-END

1 2 3

LIVE DEMO — CENTER

1 2 3

LIVE DEMO — SPACE-BETWEEN

1 2 3

LIVE DEMO — SPACE-AROUND

1 2 3

LIVE DEMO — SPACE-EVENLY

1 2 3

```
.jc-start   { justify-content: flex-start; }
.jc-end     { justify-content: flex-end; }
.jc-center  { justify-content: center; }
.jc-between { justify-content: space-between; }
.jc-around  { justify-content: space-around; }
.jc-evenly  { justify-content: space-evenly; }
```

Easy memory: `space-between` pushes the items to the edges. `space-around` puts space around each item. `space-evenly` makes every single gap identical, including the outer edges.

align-items — alignment along the cross axis

While `justify-content` handles the main axis, `align-items` handles the **cross axis** (top-to-bottom in a row). It is used to line items up vertically inside their container.

Value	What it does
`stretch`	Items stretch to fill the height (default)
`flex-start`	Items sit at the top
`center`	Items are vertically centered
`flex-end`	Items sit at the bottom
`baseline`	Items align by the baseline of their first line of text

LIVE DEMO — ALIGN-ITEMS: STRETCH (DEFAULT)

Short Normal Tall

LIVE DEMO — ALIGN-ITEMS: FLEX-START

Short Tall Normal

LIVE DEMO — ALIGN-ITEMS: CENTER

Short Tall Normal

LIVE DEMO — ALIGN-ITEMS: FLEX-END

Short Tall Normal

LIVE DEMO — ALIGN-ITEMS: BASELINE

tiny text Big Medium

```
.ai-stretch  { align-items: stretch; }
.ai-start    { align-items: flex-start; }
.ai-center   { align-items: center; }
.ai-end      { align-items: flex-end; }
.ai-baseline { align-items: baseline; }
```

align-content — spacing between multiple lines

This is the most confusing flexbox property for beginners, so read carefully. `align-content` only does something when **two conditions are both true**:

1. the container has `flex-wrap: wrap` (or wrap-reverse), and
2. the items have actually wrapped onto **more than one line**.

If there is only one line, `align-content` has nothing to do. Think of it as the property that controls the spacing between the *rows* of items — while `align-items` controls how items sit *within* a row.

LIVE DEMO — ALIGN-CONTENT: STRETCH (DEFAULT)

1 2 3 4 5 6

LIVE DEMO — ALIGN-CONTENT: FLEX-START

1 2 3 4 5 6

LIVE DEMO — ALIGN-CONTENT: CENTER

1 2 3 4 5 6

LIVE DEMO — ALIGN-CONTENT: SPACE-BETWEEN

1 2 3 4 5 6

LIVE DEMO — ALIGN-CONTENT: SPACE-AROUND

1 2 3 4 5 6

```
.ac-stretch { align-content: stretch; }
.ac-start   { align-content: flex-start; }
.ac-center  { align-content: center; }
.ac-end     { align-content: flex-end; }
.ac-between { align-content: space-between; }
.ac-around  { align-content: space-around; }
```

Quick way to remember:
`justify-content` → main axis (left/right in a row)
`align-items` → cross axis, inside each row
`align-content` → cross axis, between the rows

gap — the modern way to add space between items

Before the `gap` property existed, developers had to add margins to every item and then remove the margin from the last one. It was messy. Now you just write `gap` on the container and the browser puts even spacing between items.

LIVE DEMO — GAP: 0

1 2 3

LIVE DEMO — GAP: 8PX

1 2 3

LIVE DEMO — GAP: 30PX

1 2 3

```
.gap-0  { gap: 0; }
.gap-8  { gap: 8px; }
.gap-16 { gap: 16px; }
.gap-30 { gap: 30px; }
```

row-gap and column-gap — different spacing each way

If you want the rows and columns to have different spacing, use `row-gap` and `column-gap` separately. This is very useful for card grids, image galleries, and tag clouds.

LIVE DEMO — ROW-GAP: 30PX, COLUMN-GAP: 8PX

1 2 3 4 5 6 7 8

Notice the rows are far apart, but the items inside each row are close together.

LIVE DEMO — COLUMN-GAP: 30PX, ROW-GAP: 8PX

1 2 3 4 5 6 7 8

Here it is the opposite — wide columns, tight rows.

```
.gap-row-demo {
    row-gap: 30px;       /* space between rows */
    column-gap: 8px;     /* space between columns */
}
.gap-col-demo {
    column-gap: 30px;
    row-gap: 8px;
}
```

Bonus: `gap` also works in CSS Grid, and even in multi-column layouts. It is one of the most universally useful modern properties.

Flexbox cheat sheet

Property	Goes on	What it controls	Common values
`display`	Container	Turns flexbox on	`flex`, `inline-flex`
`flex-direction`	Container	Direction of the main axis	`row`, `column`, `row-reverse`, `column-reverse`
`flex-wrap`	Container	Whether items can wrap to new lines	`nowrap`, `wrap`, `wrap-reverse`
`flex-flow`	Container	Shortcut for direction + wrap	`row wrap`, `column nowrap`
`justify-content`	Container	Spacing along the main axis	`center`, `space-between`, `space-evenly`
`align-items`	Container	Alignment along the cross axis (inside each line)	`center`, `stretch`, `flex-start`
`align-content`	Container	Spacing between wrapped lines	`space-between`, `center`, `stretch`
`gap`	Container	Space between all items	`10px`, `1rem`
`row-gap`	Container	Space between rows only	`20px`
`column-gap`	Container	Space between columns only	`20px`

3 Advanced CSS properties

These properties are the ones that separate a plain-looking page from a polished, professional one. They handle what happens when content is too big, how the mouse behaves, how things are hidden, and how to apply beautiful visual effects like blur and colour filters.

overflow — what happens when content is too big

Sometimes the content inside a box is bigger than the box itself. Maybe there is a long paragraph in a small container, or a large image inside a thin frame. The `overflow` property decides what the browser should do in that situation.

Value	What happens
`visible`	The content spills out of the box and is fully shown (the default)
`hidden`	The extra content is cut off and completely invisible
`scroll`	Scrollbars always appear, even if they are not needed
`auto`	Scrollbars appear only when they are needed (the friendliest option)

LIVE DEMO — OVERFLOW (ALL VALUES SIDE BY SIDE)

overflow: visible

This box is only 110px tall but the text is much longer, so it spills right out of the border and overlaps whatever comes next. This is the default behaviour.

overflow: hidden

This box is only 110px tall but the text is much longer, so the extra content is simply cut off. Whatever does not fit is invisible.

overflow: scroll

This box is only 110px tall but the text is much longer, so scrollbars appear on both sides — even though you only need the vertical one.

overflow: auto

This box is only 110px tall but the text is much longer, so a scrollbar appears only where it is actually needed. This is usually the friendliest choice.

Try scrolling inside the `scroll` and `auto` boxes.

```
.visible { overflow: visible; }   /* content spills out (default) */
.hidden  { overflow: hidden; }    /* extra content is cut off */
.scroll  { overflow: scroll; }    /* scrollbars always shown */
.auto    { overflow: auto; }      /* scrollbars only when needed */
```

overflow-x and overflow-y — one direction at a time

You can also control the two directions separately. This is often used to allow horizontal scrolling in a table, but keep the vertical scrolling hidden, or vice versa.

```
.horizontal-only {
    overflow-x: auto;      /* may show a horizontal scrollbar */
    overflow-y: hidden;    /* never show a vertical scrollbar */
}

.vertical-only {
    overflow-x: hidden;
    overflow-y: auto;
}
```

Careful: if you set `overflow-x` to anything other than `visible`, and leave `overflow-y` as `visible`, the browser will silently change `visible` to `auto`. So it is best to always set both together.

text-overflow — putting "..." when text is too long

When you have a long piece of text in a narrow space — like a file name, a card title, or a table cell — you often want to cut it off with a neat "..." at the end. That is exactly what `text-overflow` does.

There is an important catch though: `text-overflow` only works when **three properties** are combined:

1. `white-space: nowrap;` — to keep everything on one line
2. `overflow: hidden;` — to hide whatever does not fit
3. `text-overflow: ellipsis;` — to add the "..."

LIVE DEMO — TEXT-OVERFLOW

text-overflow: ellipsis — adds "..." at the cut point

This is a very long line of text that will not fit inside the box.

text-overflow: clip — cuts the text off abruptly

This is a very long line of text that will not fit inside the box.

Multi-line clamp — cuts after 2 lines with ...

This is a much longer paragraph that will wrap onto several lines. Using line-clamp, only the first two lines are shown and the rest is replaced by an ellipsis. This is very common on news cards and product listings.

```
/* One-line ellipsis */
.ellipsis {
    white-space: nowrap;     /* keep it on one line */
    overflow: hidden;        /* hide the overflow */
    text-overflow: ellipsis; /* show "..." */
}

/* Simple clip instead */
.clip {
    white-space: nowrap;
    overflow: hidden;
    text-overflow: clip;     /* no "..." - just cut off */
}

/* Multi-line clamp (only the first 2 lines) */
.multi {
    display: -webkit-box;
    -webkit-line-clamp: 2;
    -webkit-box-orient: vertical;
    overflow: hidden;
}
```

cursor — changing the mouse pointer

The `cursor` property changes what the mouse pointer looks like when it hovers over an element. This seems like a small detail, but it makes a huge difference to how usable a website feels. A button that shows a hand cursor feels clickable.

LIVE DEMO — HOVER EACH BOX TO SEE THE CURSOR

default pointer text move wait help not-allowed crosshair grab zoom-in progress cell

```
.default     { cursor: default; }      /* the normal arrow */
.pointer     { cursor: pointer; }      /* the pointing hand - for buttons and links */
.text        { cursor: text; }         /* the I-beam - for text */
.move        { cursor: move; }         /* the four arrow cross - for draggable items */
.wait        { cursor: wait; }         /* the spinning circle - for loading */
.help        { cursor: help; }         /* arrow with a question mark */
.notallowed  { cursor: not-allowed; }  /* the "no entry" sign - for disabled items */
.crosshair   { cursor: crosshair; }    /* a thin cross - for drawing or selecting */
.grab        { cursor: grab; }         /* the open hand - for dragging */
.zoom        { cursor: zoom-in; }      /* magnifying glass */
.progress    { cursor: progress; }     /* another loading style */
.cell        { cursor: cell; }         /* a thick cross - for spreadsheet cells */
```

Real-world example: any element you make clickable with JavaScript should get `cursor: pointer;`. Users expect that hand cursor to mean "this is clickable".

visibility — hide something but keep its space

The `visibility` property makes an element invisible. But — and this is the important part — it still **takes up its normal space** in the layout. The element is gone visually, but the space it occupied is still reserved.

Compare this with `display: none`, which removes the element completely and lets everything else slide into its place.

Property	Visible?	Takes up space?	Best used for
`visibility: hidden`	No	Yes (a gap remains)	Hiding something temporarily without moving the layout
`display: none`	No	No (removed from layout)	Removing elements entirely — menus, popups, hidden panels

LIVE DEMO — COMPARE THE TWO

visibility: hidden — the middle box disappears but a gap is left where the hidden box was.

Before visibility: hidden After

display: none — the middle box is completely gone. The other two slide together with no gap.

Before display: none After

```
.visibility-hidden {
    visibility: hidden;    /* invisible, but still takes up space */
}

.display-none {
    display: none;         /* completely removed from the layout */
}
```

Simple way to remember: `visibility: hidden` is like a person standing behind a curtain — still there, taking up room. `display: none` is like a person who has left the room — gone completely.

filter — visual effects on any element

The `filter` property applies graphical effects like blur, colour changes, and shadows to any element — images, videos, even entire containers. It works like a photo-editing app, but written in CSS.

Filter	What it does
`blur(px)`	Softens the image
`brightness(%)`	Makes it lighter or darker
`contrast(%)`	Increases or decreases contrast
`grayscale(%)`	Removes colour
`sepia(%)`	Gives an old-photo brown tint
`hue-rotate(deg)`	Shifts all colours around the wheel
`invert(%)`	Flips colours to their opposites
`saturate(%)`	Makes colours more or less vivid
`drop-shadow()`	Adds a shadow that follows the shape of the image

LIVE DEMO — HOVER ANY IMAGE TO REMOVE THE FILTER

IMG no filter

IMG blur(2px)

IMG grayscale(100%)

IMG sepia(1)

IMG brightness(1.5)

IMG contrast(2.2)

IMG hue-rotate(180deg)

IMG invert(1)

IMG saturate(3)

IMG drop-shadow()

```
.blur       { filter: blur(2px); }
.grayscale  { filter: grayscale(1); }
.sepia      { filter: sepia(1); }
.bright     { filter: brightness(1.5); }
.contrast   { filter: contrast(2.2); }
.hue        { filter: hue-rotate(180deg); }
.invert     { filter: invert(1); }
.saturate   { filter: saturate(3); }
.dropshadow { filter: drop-shadow(4px 4px 8px rgba(0,0,0,0.4)); }
```

Combining several filters in one line

You can chain filters by separating them with spaces. They are applied from left to right.

```
.combo {
    filter: grayscale(0.4) brightness(1.2) blur(1px);
}
```

Performance tip: filters are beautiful but can be expensive to render, especially blur. Avoid applying them to very large elements or animating them continuously on low-end devices.

backdrop-filter — the "frosted glass" effect

`backdrop-filter` is a close cousin of `filter`. But instead of filtering the element itself, it filters **whatever is behind the element**. The most popular use is a blur — which creates that beautiful "frosted glass" look you see on modern operating systems and websites.

LIVE DEMO — FROSTED GLASS CARD

backdrop-filter: blur(10px)

No backdrop-filter (solid background)

The left card blurs the colourful gradient behind it. The right card has a solid background — notice how different they look.

```
.glass-card {
    background: rgba(255,255,255,0.25);   /* semi-transparent */
    backdrop-filter: blur(10px);          /* blurs what is behind it */
    -webkit-backdrop-filter: blur(10px);  /* for Safari support */
    border: 1px solid rgba(255,255,255,0.4);
    border-radius: 14px;
}
```

Other backdrop-filter effects

```
.bd-blur      { backdrop-filter: blur(8px); }
.bd-bright    { backdrop-filter: brightness(0.6); }
.bd-contrast  { backdrop-filter: contrast(0.4); }
.bd-grayscale { backdrop-filter: grayscale(1); }
.bd-saturate  { backdrop-filter: saturate(2); }
.bd-sepia     { backdrop-filter: sepia(0.7); }
.bd-combo     { backdrop-filter: blur(6px) saturate(1.8) brightness(1.1); }
```

Browser support: `backdrop-filter` works in all modern browsers, but older versions of Safari need the `-webkit-` prefix. It is a good practice to write both.

object-fit — how an image fits inside its box

This is one of the most useful properties for image galleries and card layouts. When you put an image in a box that has a different shape than the image itself, `object-fit` decides how the image should behave.

Value	What happens
`fill`	Stretches the image to fill the box exactly, even if it distorts (the default)
`contain`	Scales the image to fit entirely inside the box, keeping its proportions. May leave empty space
`cover`	Scales the image to completely cover the box, keeping proportions. May crop the edges
`none`	Shows the image at its natural size, cropping whatever does not fit
`scale-down`	Picks whichever of `none` or `contain` produces a smaller image

LIVE DEMO — OBJECT-FIT VALUES

IMG fill (squashed)

IMG contain (fully visible)

IMG cover (fills, crops)

IMG none (natural size)

IMG scale-down

All five images are the same file, placed into the same 130px tall frame. Only the `object-fit` value is different.

```
img {
    width: 100%;
    height: 140px;
}
.fill       { object-fit: fill; }        /* stretches, may distort */
.contain    { object-fit: contain; }     /* whole image visible, may letterbox */
.cover      { object-fit: cover; }       /* fills the box, crops edges */
.none       { object-fit: none; }        /* natural size, cropped */
.scale-down { object-fit: scale-down; }  /* the smaller of none / contain */
```

object-position — which part of the image stays visible

When you use `object-fit: cover`, some of the image gets cropped. The `object-position` property decides which part gets cropped away.

```
img {
    object-fit: cover;
    object-position: center;   /* keeps the middle visible (default) */
    /* object-position: top;     keeps the top visible */
    /* object-position: bottom;  keeps the bottom visible */
    /* object-position: left;    keeps the left side visible */
    object-position: 20% 80%;  /* custom position */
}
```

Real-world use: almost every modern image gallery uses `object-fit: cover;` on thumbnails. It guarantees that every card has exactly the same size, no matter what shape the original photo was.

Advanced properties cheat sheet

Property	What it does	Most useful value
`overflow`	What to do when content is bigger than the box	`auto`
`overflow-x` / `overflow-y`	Control each direction separately	`auto`
`text-overflow`	Adds "..." to cut-off text	`ellipsis`
`cursor`	Change the mouse pointer	`pointer`
`visibility`	Hide an element but keep its space	`hidden`
`filter`	Apply photo-style effects to an element	`blur()`, `grayscale()`
`backdrop-filter`	Blur or effect whatever is behind the element	`blur(10px)`
`object-fit`	How an image fits inside its box	`cover`
`object-position`	Which part of a cropped image stays visible	`center`

WHAT YOU WILL LEARN

1. How to use Google Fonts
2. Custom Fonts with @font-face
3. The BEM Naming Convention

1 How to use Google Fonts

Google Fonts is a free library with over 1,500 typefaces that anyone can use on any website. Instead of downloading font files and hosting them yourself, you just add one link to your HTML — and the browser fetches the fonts from Google's super-fast servers.

Think of it like ordering food delivery instead of cooking. You don't need to own the kitchen — you just place one order (one link), and the meal (the font) arrives at your page ready to use.

Why Google Fonts is popular: it is free, fast, reliable, and works on almost every device in the world. No license fees, no hosting worries, no maintenance.

Step 1 — Pick a font on fonts.google.com

Go to `fonts.google.com`, browse the fonts, and click on any one you like. You will see buttons for different weights (like Regular 400, Medium 500, Bold 700) and different styles (normal and italic). Click "Select this style" for each one you want.

Step 2 — Copy the <link> code

Google gives you a ready-made code block. Paste it into the `<head>` section of your HTML file. It looks like this:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;700&family=Playfair+Display:wght@700&display=swap" rel="stylesheet">
```

Let's break this down line by line:

- **preconnect** — the first two lines tell the browser "hey, we are going to load something from these servers soon, so open the connection now". This saves valuable time. The second also includes the `crossorigin` attribute required for the font connection.
- **The stylesheet link** — this is the actual font. The `family=Inter:wght@400;700` part means "give me the Inter font in weight 400 and weight 700".
- **display=swap** — this is very important. It tells the browser "if the font hasn't loaded yet, show the text with a fallback font first, then swap in the real one when it arrives". Without this, your text might be invisible for a moment.

Step 3 — Use the font in your CSS

Now that the font is loaded, you use it just like any other font. The name is the same one shown on Google Fonts (usually in title case). If the name has spaces in it, quote it (it doesn't hurt otherwise).

```
body {
    font-family: 'Inter', sans-serif;
}

.special {
    font-family: 'Playfair Display', serif;
}
```

Always add a fallback! After the Google Font name, put a comma and then a generic family like `sans-serif` or `serif`. That way, if the Google Font fails to load for any reason, the page still looks reasonable.

Live Demo — Four Google Fonts side by side

This page itself is loading four different Google Fonts. Here is what each one looks like:

LIVE DEMO — GOOGLE FONTS IN ACTION

Inter

The quick brown fox jumps over the lazy dog. Inter is clean, modern, and highly readable.

Playfair Display

The quick brown fox jumps over the lazy dog. Playfair is elegant and classy — perfect for headlines.

Roboto Mono

The quick brown fox jumps over the lazy dog. Roboto Mono is fixed-width — great for code.

Pacifico

The quick brown fox jumps over the lazy dog. Pacifico is a playful handwritten script.

Loading different weights of the same font

Many fonts come in different **weights** — thin, light, regular, medium, semi-bold, bold, and black. Each weight is a separate file, so you should only load the ones you actually use.

LIVE DEMO — FIVE WEIGHTS OF INTER

Weight 200 — light and elegant

Weight 400 — the normal reading weight

Weight 600 — slightly bolder

Weight 700 — the classic bold

Weight 900 — heavy and strong

```
/* Load weights 200, 400, 600, 700, 900 of Inter in one link */
<link href="https://fonts.googleapis.com/css2?family=Inter:wght@200;400;600;700;900&display=swap" rel="stylesheet">

/* Then use them in your CSS */
.light  { font-family: 'Inter', sans-serif; font-weight: 200; }
.normal { font-family: 'Inter', sans-serif; font-weight: 400; }
.medium { font-family: 'Inter', sans-serif; font-weight: 600; }
.bold   { font-family: 'Inter', sans-serif; font-weight: 700; }
.heavy  { font-family: 'Inter', sans-serif; font-weight: 900; }
```

The @import method (alternative)

Instead of the `<link>` tag, you can import a Google Font from inside your CSS file. Put it at the very top of the CSS, so it is shown before the link method because the browser has to download the CSS first and only then discover the font.

```
/* Put this at the very top of your CSS file */
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;700&display=swap');
```

Best practice: always prefer the `<link>` method over `@import`. The link method is faster because the browser starts downloading the font earlier, in parallel with the CSS.

Loading different styles — normal and italic

If you want both normal and italic versions of a font, you list them in the URL. The `0,` means "normal style" and the `1,` means "italic style".

```
/* Load normal AND italic versions of Playfair Display */
<link href="https://fonts.googleapis.com/css2?family=Playfair+Display:ital,wght@0,400;1,400&display=swap" rel="stylesheet">
```

LIVE DEMO — NORMAL VS ITALIC PLAYFAIR DISPLAY

This text is normal Playfair Display

This text is italic Playfair Display

Best practices for Google Fonts

Do this	Why
Use `preconnect` for `fonts.googleapis.com` and `fonts.gstatic.com`	Opens the connection early, saving 100–300ms
Always add `display=swap`	Text becomes visible immediately instead of invisible text
Limit to 2–3 fonts	Each font adds more time to load your page
Only load the weights you actually use	Each weight is a separate file — fewer files = faster page
Combine multiple fonts into one link	One request instead of several
Use a fallback like `sans-serif` or `serif`	Makes the page readable even if the font fails to load

Which font should you pick?

Category	Fonts like	Best for	Examples
Sans-serif	Modern, clean, neutral	Body text, UI, apps	Inter, Roboto, Open Sans, Poppins
Serif	Classic, elegant, traditional	Headings, blogs, magazines	Playfair Display, Lora, Merriweather
Monospace	Technical, precise	Code, numbers, tables	Roboto Mono, JetBrains Mono, Fira Code
Display / Script	Decorative, expressive	Logos, big headlines, invitations	Pacifico, Lobster, Bebas Neue

A simple rule of thumb: use one sans-serif font for the body, and one serif or display font for headlines. Two fonts are enough for almost any website.

2 Custom Fonts with @font-face

Sometimes the perfect font is not on Google Fonts. Maybe you bought it, or your designer gave you a file, or you just want to use a font that only exists on your computer. In that case, you use the `@font-face` rule to tell the browser "here is the font file, please use it".

`@font-face` is like making a guest list for a party. You give the browser a name for the font (like `"MyCustomFont"`) and a link to the file. From that moment on, you can use `"MyCustomFont"` anywhere in your CSS just like any other font.

The big difference: with Google Fonts, someone else hosts the font file. With `@font-face`, you host the font file on your own server. This means you control everything — but you also have to upload the files yourself and worry about licenses.

The basic syntax

```
@font-face {
    font-family: 'MyCustomFont';            /* the name you will use later */
    src: url('fonts/MyFont.woff2') format('woff2'),
         url('fonts/MyFont.woff')  format('woff');
    font-weight: 400;                       /* normal weight */
    font-style: normal;                     /* normal (not italic) */
    font-display: swap;                     /* show fallback first, then swap */
}
```

Let's break down every line:

- **font-family** — any name you want. This is the name you will use in other rules. It does not have to match the file name.
- **src** — the path to the font file. You can list several formats; the browser uses the first one it understands. `format()` tells the browser what type of file it is.
- **font-weight** — which weight this particular file represents (for example, 700 for bold).
- **font-style** — `normal` or `italic`. If you have a separate italic file, declare it in a second `@font-face` block.
- **font-display** — what to do while the font is downloading. `swap` is usually the friendliest option.

Live Demo — this page loads a real @font-face

The paragraph below uses a custom font loaded with `@font-face`. It is the same Pacifico font you saw earlier, but here we are loading it directly by its file URL, not through Google's stylesheet.

LIVE DEMO — @FONT-FACE LOADED FONT

Hello, I am loaded with @font-face!

```
@font-face {
    font-family: 'MyHandwriting';
    src: url('https://fonts.gstatic.com/s/pacifico/v22/FwZY7-Qmy14u9lezJ-6H6MmBp0u-.woff2') format('woff2');
    font-weight: 400;
    font-style: normal;
    font-display: swap;
}

.ff-demo {
    font-family: 'MyHandwriting', cursive;
}
```

Font formats — which one should you use?

Browsers understand several different font file formats. Each one has its own history and its own trade-offs.

Format	File extension	Size	Support
WOFF2	`.woff2`	Smallest ✅	All modern browsers
WOFF	`.woff`	Small	All modern browsers
TrueType	`.ttf`	Large	Almost everything
OpenType	`.otf`	Large	Almost everything
Embedded OpenType	`.eot`	Large	Only very old Internet Explorer
SVG Font	`.svg`	Very large	Almost dead — do not use

Simple rule: modern best practice is to provide **WOFF2** and **WOFF** only. WOFF2 is smaller and faster, and WOFF is a safe backup. You no longer need TTF, OTF, EOT, or SVG anymore.

Providing several formats at once

```
@font-face {
    font-family: 'MyFont';
    src: url('MyFont.woff2') format('woff2'),   /* best - try first */
         url('MyFont.woff')  format('woff'),    /* fallback */
         url('MyFont.ttf')   format('truetype'); /* oldest fallback */
    font-weight: 400;
}
```

The browser reads the list from top to bottom and stops at the first format it understands. So always put WOFF2 first — because it has the best compression — and browsers that can handle it will download the lighter files.

Loading multiple weights of the same custom font

If you have several font files for the same family (Regular, Bold, Italic), you write a separate `@font-face` block for each one. Use the **same** `font-family` name, but different `font-weight` and `font-style`.

```
/* Regular */
@font-face {
    font-family: 'MyFont';
    src: url('fonts/MyFont-Regular.woff2') format('woff2');
    font-weight: 400;
    font-style: normal;
}

/* Bold */
@font-face {
    font-family: 'MyFont';
    src: url('fonts/MyFont-Bold.woff2') format('woff2');
    font-weight: 700;
    font-style: normal;
}

/* Italic */
@font-face {
    font-family: 'MyFont';
    src: url('fonts/MyFont-Italic.woff2') format('woff2');
    font-weight: 400;
    font-style: italic;
}

/* Now use it normally */
p { font-family: 'MyFont', sans-serif; }
strong { font-weight: 700; }             /* uses the bold file */
em { font-style: italic; }               /* uses the italic file */
```

The browser is smart enough to pick the right file automatically based on the `font-weight` and `font-style` of the element.

font-display — what to show while the font loads

When someone visits your site for the first time, the custom font has to download. During that brief moment — usually less than a second, but sometimes more on slow connections — the browser has to decide what to show. That decision is made by `font-display`.

Value	What happens	Best for
`auto`	Whatever the browser decides (usually block)	Not recommended
`block`	Invisible text for up to 3 seconds, then the font appears	Rarely — only if text must be the custom font
`swap`	Fallback font shows immediately, swaps when it loads	Body text ✅
`fallback`	Very short invisible time, then fallback if font is slow	A balance between block and swap
`optional`	Uses fallback if the font hasn't loaded almost immediately	Performance-focused sites

LIVE DEMO — THE FOUR FONTS USED IN THIS PAGE

Playfair Display

Inter

Roboto Mono

Pacifico

Notice how different fonts give completely different personalities.

What if the font fails to load?

Always, always provide a **fallback stack**. If your custom font fails to load — the file is missing, the server is down, or the user is offline — the browser will use the next font in the list.

LIVE DEMO — FALLBACK IN ACTION

The font from this font does not exist, so I fall back to Playfair Display.

```
/* This is the code for the paragraph above */
.ff-fallback {
    font-family: 'ThisFontDoesNotExist', 'Playfair Display', serif;
}
```

The browser tries `'ThisFontDoesNotExist'`, finds nothing, then falls back to `'Playfair Display'`, which is loaded on the page.

Using local() to skip the download

If the user already has the font installed on their computer, you can use `local()` to tell the browser to try the local version first, and only download if it can't find it.

```
@font-face {
    font-family: 'MyFont';
    src: local('MyFont Regular'),              /* try local first */
         local('MyFont-Regular'),              /* alternative local name */
         url('MyFont.woff2') format('woff2');   /* download if not found */
}
```

When to use local(): mostly for system fonts or very common fonts like Helvetica or Arial. For custom brand fonts, it is almost never going to be a local match — it saves bandwidth for users who already have it.

Legal note — you must own the license

Very important: most commercial fonts (like Helvetica, Futura, Proxima Nova) are licensed per website. Just because you have the file on your computer does not mean you can upload it to your website. Always check the license. Google Fonts is safe because every font there is explicitly free for web use. Other free sources include Font Squirrel and the Open Font Library.

Where to get free custom fonts

- **Google Fonts** — fonts.google.com (1,500+ free fonts)
- **Font Squirrel** — fontsquirrel.com (curated free fonts)
- **Fontshare** — fontshare.com (free from Indian Type Foundry)
- **The League of Moveable Type** — theleagueofmoveabletype.com (open-source fonts)

@font-face cheat sheet

Property	What it does	Example
`font-family`	Names your font	`'MyFont'`
`src`	Where the file is	`url('x.woff2') format('woff2')`
`font-weight`	Which weight the file is	`400`, `700`
`font-style`	Normal or italic	`normal`, `italic`
`font-display`	What to show while loading	`swap`, `block`, `optional`
`unicode-range`	Only load certain characters (advanced)	`U+0000-00FF`

3 The BEM Naming Convention

Imagine a project with 200 CSS classes. Some are called `.red`, some are called `.button-new`, some are just `.box`. Nothing tells you where a class is used, what it is for, or whether it is safe to change it. This is how most CSS projects turn into chaos over time.

BEM is a naming system that stops this chaos. BEM stands for **Block, Element, Modifier**. It gives every class a clear, predictable name that tells you exactly what it is and where it belongs.

The core idea: every CSS class should answer two questions — what is it (the block), what part of it is (the element), and how is this one different from the others (the modifier)?

The three parts of BEM

Block — a standalone component

A **block** is a self-contained piece of the page. It could be a card, a menu, a form, a header, a button — anything that makes sense on its own. Blocks are named with simple lowercase words separated by hyphens.

.card .menu .search-form .user-profile .site-header

Element — a part inside a block

An **element** is a piece *inside* a block. It does not make sense on its own — a "title" only means something if you know it's the title *of* a card. Elements are joined to their block with **two underscores** `__`.

.card__title .card__image .card__button .menu__item .menu__link

Modifier — a variation of a block or element

A **modifier** is a version of a block or element that looks slightly different. A "featured card" is still a card, but with a golden border. A "primary button" is still a button, but with a bold colour. Modifiers are joined with **two hyphens** `--`.

.card--featured .card--small .card__button--primary .card__button--secondary .menu__item--active .menu__item--disabled

Reading a BEM name

Once you know the pattern, any BEM class name tells you its story:

```
.card                      → Block. The whole card component.
.card__title               → Element. The title inside the card.
.card__title--large        → Modifier. A big version of the card title.
.card__button--primary     → Modifier. A primary (highlighted) button.
.user-profile__avatar      → Element. The avatar inside the profile block.
.menu__item--active        → Modifier. The currently active menu item.
```

The three symbols to remember

Symbol	Meaning	Example
Nothing between words	Just a block name (use hyphens for multiple words)	`.search-form`
`__` (two underscores)	Element — part of a block	`.search-form__input`
`--` (two dashes)	Modifier — variation	`.search-form--dark`

Easy memory: think of it as a house. *Block* = the house. *Element* = a room inside the house (kitchen, bedroom). *Modifier* = what kind of kitchen (modern, rustic). You would never say "kitchen" without mentioning which house it belongs to — that is why the name always keeps the block name: `house__kitchen--modern`.

Live Demo — a real card built with BEM

Here is a product card whose classes all follow BEM. You can see the class names listed below each part — every one of them follows the pattern `block__element--modifier`.

LIVE DEMO — BEM CARD COMPONENT

Product

Sunset Lamp

A warm gradient lamp that fills your room with amber light.

₹1,499 Add to cart

Featured

Golden Lamp

Our best-selling lamp with a hand-painted golden finish.

₹2,999 Limited stock

Notice how the **modifiers** above only change what they need to change: `.bem-card--featured` adds a golden border and a different gradient image, but everything else stays identical to the normal card. This is the real power of BEM — you make small variations without duplicating all the base styles.

The BEM code in action

```
/* ---------- BLOCK ---------- */
.bem-card {
    background: #ffffff;
    border-radius: 12px;
    box-shadow: 0 4px 14px rgba(0,0,0,0.08);
    overflow: hidden;
}

/* ---------- ELEMENTS ---------- */
.bem-card__image {
    height: 100px;
    background: linear-gradient(135deg, #fbbf24, #f43f5e);
}

.bem-card__body {
    padding: 14px 16px;
}

.bem-card__title {
    font-size: 1.1rem;
    font-weight: 800;
    margin-bottom: 4px;
}

.bem-card__text {
    font-size: 0.9rem;
    color: #475569;
    margin-bottom: 10px;
}

.bem-card__price {
    font-family: 'Roboto Mono', monospace;
    font-size: 1.2rem;
    font-weight: 700;
    color: #0f172a;
}

.bem-card__button {
    background: linear-gradient(135deg, #6d28d9, #3b82f6);
    color: #fff;
    padding: 8px 16px;
    border: none;
    border-radius: 8px;
    font-weight: 700;
}

/* ---------- MODIFIERS ---------- */
.bem-card--featured {
    border: 3px solid #fbbf24;
    box-shadow: 0 8px 22px rgba(251,191,36,0.35);
}

.bem-card__title--large {
    font-size: 1.4rem;
}

.bem-card__button--secondary {
    background: #f1f5f9;
    color: #0f172a;
}
```

Bad naming vs BEM naming — side by side

❌ **Bad — confusing names**

.red-title
.box1
.big-btn
.btn2
.header-link
.menu-item-2
.card-title
.card-image-big
.product-card-price

✅ **Good — BEM names**

.card__title--large
.card__button--primary
.card__button--secondary
.menu__item
.menu__item--active
.card__image
.product-card__price

The bad names tell you nothing about the context of the element. Is `.red-title` used in a card, a menu, or a footer? You would have to search the whole codebase to find out. The BEM names are self-explanatory — you can guess exactly what each one is just by reading its name.

Rules and best practices

Rule	Good ✅	Bad ❌
Use lowercase	`.card__title`	`.Card__Title`
Separate words with a single hyphen	`.user-profile__avatar`	`.userProfile_avatar`
Never nest more than one level	`.card__title`	`.card__header__title`
Elements belong to their block, not to each other	`.card__button`	`.card__footer__button`
Only use classes — no ID or tag styling	`class="card__title"`	`#main-card h2`
Each element can also have its own modifier	`.card__title` and `.card__title--red`	`.card-red`

The nesting rule in plain words: you never write `.block__element__subelement`. If you feel tempted to nest deeper, just make a new block inside the first one. For example, instead of `.card__footer__button`, make a block called `.card-footer` and call it `.card-footer__button`.

Why BEM is worth learning

- **Predictable names** — anyone opening your project can figure out what each class does just by reading it.
- **No more naming conflicts** — `.card__title` can never accidentally clash with `.menu__title`, because the prefix is unique.
- **Easier to maintain** — when you delete a block, you can safely delete every class starting with that block name. No orphan styles left.
- **Better teamwork** — when several developers work on the same project, BEM keeps everyone using the same rules.
- **Smaller CSS** — because modifiers only add what changes, you avoid duplicating huge blocks of styles.
- **Scales beautifully** — BEM works just as well for a 5-page site as it does for a 5,000-page one.

When to use BEM

BEM is a convention, not a law. It is very useful for medium to large projects, but for tiny one-page sites it can feel over-engineered. Also, if you use modern tools like CSS Modules or Tailwind CSS, you may not need BEM at all — you'll be told how they already handle scoping for you.

Final takeaway: BEM is not about making your code shorter — it is about making it clearer. A few extra characters in your class name will save you hours of confusion later. Consistency beats cleverness every single time.

BEM cheat sheet

Part	Symbol	What it means	Example
Block	— only	A standalone component	`.card`
Element	`__` (double underscore)	A part inside a block	`.card__title`
Modifier	`--` (double dash)	A variation	`.card--featured`
Multi-word names	`-` (single hyphen)	Combine words inside any part	`.user-profile__avatar-image`